Redis 학습 노트

정리일 2026-11-13 · Week11-12 AI 활용

Redis가 생긴 이유와 캐시 동작을 이해하고, 미니 DBMS를 만든 뒤 Redis를 연결하는 흐름을 정리한다.

1. Redis가 생긴 이유와 하는 일

2009년 Salvatore Sanfilippo는 실시간 방문 분석 서비스 LLOOGG를 개발하고 있었다. 페이지를 열 때마다 MySQL에 방문 기록을 쓰는 구조라 방문자가 늘면 쓰기 부하가 커졌다. 최근 방문 기록을 빠르게 가져오는 기능도 필요했다. Redis는 이 작업을 처리하기 위해 시작됐다.

Redis는 데이터를 주로 RAM에 저장하고 키로 읽거나 변경하는 서버 프로그램이다. 백엔드가 Redis에 명령을 보내는 연결에서는 백엔드가 클라이언트이고, Redis가 서버다. RAM에서 값을 바로 읽는 이점과 캐시로 원본 DB 조회를 줄이는 이점은 각각 저장 위치와 요청 경로에서 생긴다.

용도	Redis가 하는 일
상품 캐시	자주 요청하는 상품 조회 결과를 일정 시간 보관한다.
로그인 세션	세션 ID에 연결된 로그인 정보를 보관하고 만료시킨다.
조회수·요청 횟수	`INCR`로 값을 올리고, 필요하면 TTL로 집계 기간을 정한다.
최근 방문 기록	`List`의 앞에 기록을 넣고 최신 N개를 가져온다.
게임 랭킹	`Sorted Set`에 사용자별 점수를 넣고 점수 순으로 읽는다.

2. 키와 값, 명령이 오가는 과정

`SET post:42:views 10`의 키는 `post:42:views`, 저장되는 값은 문자열 `"10"`이다. `GET`은 키의 값을 읽고 `DEL`은 키를 지운다. `INCR`는 정수로 해석할 수 있는 값을 1 올린다.

```
SET post:42:views 10
INCR post:42:views
INCR post:42:views
GET post:42:views
```

두 번의 `INCR`는 각각 정수 `11`과 `12`를 돌려준다. 마지막 `GET`의 결과는 문자열 `"12"`다.

TCP는 클라이언트와 Redis 사이에서 바이트가 오가는 연결이다. **RESP**는 그 바이트가 명령, 인자, 응답 중 무엇을 뜻하는지 표현하는 형식이다.

```
백엔드 또는 redis-cli → TCP로 RESP 명령 전송
Redis → 명령과 인자 해석 → RAM의 값 읽기 또는 변경 → RESP로 응답
```

3. 캐시 히트·미스와 TTL

상품 페이지를 조회할 때 **백엔드가** Redis와 DBMS에 각각 요청을 보낸다. Redis에 요청한 키의 유효한 값이 있으면 **히트**, 없거나 만료됐으면 **미스**다. 백엔드가 캐시를 확인한 뒤 미스일 때 원본을 읽고 채우는 패턴을 **캐시 어사이드(cache-aside)**라고 한다.

```
사용자 → 백엔드 → Redis에 GET product:42
  히트 → Redis의 상품 정보로 응답
  미스 → 백엔드가 DBMS에 요청 → DBMS가 원본 상품 조회
      → 백엔드가 Redis에 SET → 사용자에게 응답
```

SET product:42 "상품 정보" EX 60은 이 키에 60초의 유효기간을 준다. **TTL**은 만료까지 남은 시간이다. 추가 변경이 없다면 30초 뒤에는 값이 나오고, 70초 뒤에는 값이 없다. 중간에 `GET`해도 만료 시각은 그대로다.

- **만료 후 같은 키에 저장**하면 그 키에 새 값이 들어간다. 만료된 키의 `GET`은 값 없음으로 처리하므로, 백엔드가 먼저 `DEL`할 필요는 없다. 원본 DB의 ID `42`와 Redis 키 `product:42`는 각 저장소에서 따로 관리한다.
- **DB 가격 변경 직후**에는 TTL이 남은 이전 가격이 보일 수 있다. 기본 흐름은 DB 수정을 성공시킨 뒤 Redis에서 `DEL product:42`로 기존 캐시를 비우는 것이다. 다음 조회가 원본에서 새 값을 가져온다.

4. 동시 요청과 재시작

- 두 백엔드가 모두 `GET`으로 `10`을 읽은 다음 각각 `SET`으로 `11`을 쓰면 최종 값은 `11`이다. 두 요청 중 하나의 증가가 덮어써진다.
- 두 백엔드가 `INCR`를 한 번씩 보내면 최종 값은 `12`다. Redis가 각 증가를 **한 명령으로 원자적으로 처리**해 10 → 11 → 12가 된다.

영속성 방식	재시작 후 동작
저장 끔	RAM에만 있던 값은 사라진다. 상품 캐시는 다음 미스에 DB에서 다시 채울 수 있다.
RDB	특정 시점의 데이터 스냅샷을 저장하고 시작할 때 읽는다.
AOF	데이터에 영향을 주는 쓰기 명령을 기록하고 시작할 때 적용해 상태를 복원한다.

RDB와 AOF를 함께 쓸 수 있다. 저장 대상의 크기는 메모리 용량을 고려하고, 장애 시 어느 시점까지 복원되는지는 디스크 저장 설정에 따라 달라진다.

5. 미니 DBMS를 먼저 만들고 Redis 연결하기

DB는 저장된 데이터, **DBMS**는 저장·조회·수정을 처리하는 소프트웨어, **SQL**은 관계형 DBMS에 요청을 표현하는 언어다. 여러 프로그램이 데이터를 찾고 함께 수정하거나 장애 뒤 복구하는 작업을 공통으로 처리하려고 DBMS가 발전했다. 1960년대 IBM IMS는 아폴로 부품 관리에 쓰였고, 1970년 에드거 코드의 관계형 모델은 물리적 저장 위치를 몰라도 데이터를 조회하는 바탕이 됐다.

관계형 DBMS는 보통 **SQL 해석 → 실행 계획 → 결과 반환** 순으로 조회한다. 인덱스는 필요한 행을 효율적으로 찾는 데 쓰인다. 트랜잭션은 주문 저장과 재고 차감처럼 함께 성공해야 하는 변경을 묶는다.

미니 DBMS와 미니 Redis는 **JavaScript + Node.js**를 구현 후보로 잡았다. 과제 원문에 언어 또는 필수 기능 조건이 있다면 그 조건에 맞춘다.

1. **미니 DBMS.** `id`, `name`, `price`를 가진 상품을 파일에 저장한다. ID 중복은 오류, `findById(42)`에 상품이 없으면 `null`로 정한다. 가격 수정 뒤 프로그램을 재시작해 새 가격을 다시 읽을 수 있는지 확인한다.
2. **미니 Redis.** `Map`으로 `SET`, `GET`, `DEL`과 TTL을 구현한다. `redis-cli` 호환이 필요하면 TCP 서버에 RESP 요청·응답 처리를 추가한다.
3. **백엔드에서 연결.** 캐시 미스가 나면 백엔드가 `findById(42)`를 호출하고 조회 결과를 Redis에 넣는다. 같은 상품을 두 번 조회하고 TTL이 지난 뒤 한 번 더 조회해, DBMS 호출이 첫 요청과 만료 후 요청에서 일어나는지 확인한다.

```
구현 순서  미니 DBMS → 미니 Redis → 백엔드에서 연결
조회 순서  백엔드 → Redis 먼저 확인 → 미스일 때 백엔드가 DBMS 조회
```

참고 자료

- Redis 창시자 인터뷰 · Redis 소개
- 캐시 어사이드 · INCR · RESP · RDB와 AOF
- IBM IMS 역사 · 관계형 모델 · PostgreSQL 쿼리 처리
- Node.js 파일 API · Node.js 네트워크 API